Projet — Mini-analyseur de texte

Méthodologie de programmation — projet fondé sur les notions étudiées jusqu'à la semaine 4

LIVRABLE

1 fichier Python

NOTIONS CLÉS

for · while · fonctions · str

OBJECTIF

Programme interactif complet

Parcours recommandé

Testez chaque fonction séparément avant de construire le menu final.

- [ ] Normaliser le texte
- [ ] Compter une lettre
- [ ] Compter les voyelles
- [ ] Compter les mots
- [ ] Inverser le texte
- [ ] Tester un palindrome
- [ ] Afficher les statistiques
- [ ] Construire le menu interactif

0 / 8 étapes

PROJET GUIDÉ

Projet — Mini-analyseur de texte

Objectif

L'objectif de ce projet est de construire un petit programme Python capable d'analyser une chaîne de caractères saisie par l'utilisateur.

Le projet doit mobiliser uniquement les notions étudiées jusqu'à présent :

- variables et types simples ;
- `input()` et `print()` ;
- opérations et comparaisons ;
- conditions `if / elif / else` ;

- boucles `for` et `while` ;
- fonctions, paramètres et `return` ;
- chaînes de caractères (`str`) ;
- `len()`, indexation et slicing ;
- opérateurs `in` / `not in` ;
- méthodes de chaînes vues en cours : `lower()`, `upper()`, `strip()`, `replace()`, `startswith()`, `endswith()`, `count()`.

Le programme ne doit pas utiliser de listes, dictionnaires, classes, pandas ou bibliothèques externes.

---

1. Principe général

L'utilisateur saisit un texte.

Exemple :

```text
Entrez un texte : Bonjour Python, bonjour programmation !
```

Le programme propose ensuite plusieurs analyses du texte à l'aide d'un menu.

```text
--- MINI-ANALYSEUR DE TEXTE ---

1 - Afficher le texte normalisé
2 - Afficher les statistiques générales
3 - Compter une lettre
4 - Compter les voyelles
5 - Inverser le texte
6 - Tester si un mot est un palindrome
0 - Quitter
```

Votre choix :
```

Le menu doit être affiché plusieurs fois jusqu'à ce que l'utilisateur choisisse `0`.

---

## 2. Fonction `normalize_text(text)`

Écrire une fonction :

```python
def normalize_text(text):
    ...
```

Cette fonction doit :

1. supprimer les espaces au début et à la fin avec `strip()` ;
2. convertir le texte en minuscules avec `lower()` ;
3. remplacer les doubles espaces par des espaces simples.

Pour supprimer plusieurs espaces successifs, vous pouvez utiliser :

```python
while "  " in text:
    text = text.replace("  ", " ")
```

La fonction doit retourner le texte normalisé avec `return`.

---

3. Fonction `count_letter(text, letter)`
```

Écrire :

```python
def count_letter(text, letter):
    ...
```

La fonction doit parcourir le texte avec une boucle `for` et compter combien de fois `letter` apparaît.

Le comptage ne doit pas dépendre des majuscules/minuscules.

Exemple :

```python
count_letter("Banana", "a")
```

doit retourner `3`.

---

4. Fonction `count_vowels(text)`

Écrire :

```python
def count_vowels(text):
    ...
```

Elle doit compter le nombre total de voyelles.

Pour ce projet, on considère :

```python
```

"aàâäeéèêëiîïoôöuùûüy"
```

Vous pouvez tester l'appartenance avec `in`.

---

## 5. Fonction `count_words(text)`

Écrire :

```python
def count_words(text):
    ...
```

On considère que les mots sont séparés par des espaces.

Après normalisation :

- si le texte est vide, retourner `0` ;
- sinon, le nombre de mots est égal au nombre d'espaces + 1.

Exemple : `"bonjour tout le monde"` contient 3 mots.

---

6. Fonction `reverse_text(text)`

Écrire :

```python
def reverse_text(text):
    ...
```
```

```

Elle doit retourner le texte dans l'ordre inverse.

Exemple :

```python
reverse_text("Python")
```

retourne :

```text
nohtyP
```

Vous pouvez utiliser :

```python
text[::-1]
```

Pour aller plus loin

Écrire une seconde version avec une boucle, sans utiliser `[::-1]`.

---

7. Fonction `is_palindrome(word)`

Un palindrome se lit de la même manière dans les deux sens.

Exemples : `radar`, `kayak`, `elle`.

Écrire :
```

```python
def is_palindrome(word):
    ...
```

La fonction doit :

1. supprimer les espaces inutiles ;
2. convertir en minuscules ;
3. comparer le mot avec sa version inversée ;
4. retourner `True` ou `False`.

`is_palindrome("Radar")` doit retourner `True`.

---

8. Statistiques générales

Créer :

```python
def display_statistics(text):
    ...
```

Elle doit afficher au minimum :

- le nombre total de caractères ;
- le nombre de caractères sans compter les espaces ;
- le nombre de mots ;
- le nombre de voyelles ;
- le nombre d'occurrences de la lettre `a`.

Pour calculer le nombre de caractères sans espaces, utilisez une boucle et un compteur.

---

9. Programme principal : menu interactif

Le programme principal doit utiliser une boucle `while`.

Structure possible :

```python
choice = ""

while choice != "0":

    print("1 - Texte normalisé")
    print("2 - Statistiques")
    print("3 - Compter une lettre")
```

```
    print("4 - Compter les voyelles")
    print("5 - Inverser le texte")
    print("6 - Tester un palindrome")
    print("0 - Quitter")

    choice = input("Votre choix : ")

    if choice == "1":
        ...
    elif choice == "2":
        ...
    elif choice == "3":
        ...
```

Pour l'option 3, demander la lettre à compter.

Pour l'option 6, demander le mot à tester.

Si l'utilisateur choisit une option inconnue, afficher un message d'erreur puis reproposer le menu.

---

10. Organisation du programme

Structure conseillée :

```python
# Fonctions

def normalize_text(text):
    ...

def count_letter(text, letter):
    ...
```

```
def count_vowels(text):
    ...

def count_words(text):
    ...

def reverse_text(text):
    ...

def is_palindrome(word):
    ...

def display_statistics(text):
    ...


# Programme principal

text = input("Entrez un texte : ")

# menu
...
```

Évitez de recopier le même code plusieurs fois : réutilisez vos fonctions.

---

11. Vérifications

Test 1 — normalisation

Entrée :

```text
```

```
  Bonjour   PYTHON
```

Résultat attendu :

```text
bonjour python
```

Test 2 — comptage

`count_letter("Banana", "a")` doit retourner `3`.

Test 3 — palindrome

`is_palindrome("Radar")` doit retourner `True`.

Test 4 — texte vide

`count_words("")` doit retourner `0`.

Test 5 — menu

Vérifier que :

- plusieurs choix successifs sont possibles ;
- le programme ne s'arrête pas après une seule opération ;
- il quitte uniquement avec `0`.

---

12. Contraintes

Obligatoire

Le projet doit utiliser :

- au moins une boucle `for` ;
- au moins une boucle `while` ;
- des conditions ;
- plusieurs fonctions avec paramètres ;
- au moins une fonction avec `return` ;
- des opérations sur les chaînes de caractères.

Interdit

Ne pas utiliser :

- listes ;
- dictionnaires ;
- ensembles ;
- classes ;
- pandas ;
- NumPy ;
- bibliothèques NLP ;
- bibliothèques externes.

L'objectif est de résoudre le problème uniquement avec les notions vues en cours.

---

13. Bonus

Bonus A — consonnes

Créer `count_consonants(text)`.

Bonus B — début et fin

Permettre de vérifier si le texte commence ou se termine par une chaîne donnée avec `startswith()` et `endswith()`.

Bonus C — recherche

Demander une chaîne et indiquer :

- si elle apparaît ;
- combien de fois elle apparaît.

Bonus D — nettoyage simple

Créer une fonction qui supprime quelques signes de ponctuation avec `replace()` :

```text
. , ! ? ; :
```

Bonus E — statistiques sur une lettre

Pour une lettre saisie :

- nombre d'occurrences ;
- présence ou absence ;
- proportion approximative dans le texte.

---

14. Livrable

À rendre :

```text
projet_texte.py
```

Le programme doit :

1. s'exécuter avec Python 3 ;

2. ne provoquer aucune erreur sur les tests proposés ;
3. être lisible et correctement indenté ;
4. utiliser des noms explicites ;
5. contenir quelques commentaires utiles.

---

15. Barème indicatif (/20)

- **4 pts** — `normalize_text()`, `count_letter()`, `count_vowels()`, `count_words()` ;
- **3 pts** — `reverse_text()` et `is_palindrome()` ;
- **3 pts** — statistiques générales ;
- **4 pts** — menu interactif avec `while` et conditions ;
- **3 pts** — bonne utilisation des fonctions, paramètres et `return` ;
- **2 pts** — lisibilité et organisation ;
- **1 pt** — tests et gestion simple des saisies incorrectes.

Les bonus peuvent compenser de petites erreurs, sans dépasser 20/20.

Conseil

Construisez le programme étape par étape.

Testez chaque fonction séparément avant de construire le menu principal :

```python
print(normalize_text("  Bonjour  PYTHON  "))
print(count_letter("banana", "a"))
print(count_vowels("programmation"))
print(count_words("bonjour tout le monde"))
print(reverse_text("Python"))
print(is_palindrome("radar"))
```

```

## README et utilisation des LLM — obligatoire

Vous devez rendre deux fichiers :

`projet_texte.py`

`README.md`

### Contenu du README

Le fichier `README.md` doit expliquer brièvement :

- comment exécuter le programme ;
- les fonctionnalités réalisées ;
- les éventuels bonus réalisés ;
- les difficultés rencontrées ;
- les choix importants faits pendant le développement.

### Déclaration obligatoire de l'usage des LLM

Le README doit contenir une section intitulée `## Utilisation d'un LLM`.

Si vous avez utilisé ChatGPT, Claude, Gemini, Copilot, Mistral ou un autre LLM, indiquez :

- le nom de l'outil utilisé ;
- le modèle ou la version, si vous la connaissez ;
- les tâches pour lesquelles le LLM a été utilisé ;
- les prompts ou demandes principales ;
- les parties proposées par le LLM que vous avez conservées ;
- les modifications que vous avez faites vous-même ;
- comment vous avez vérifié le code obtenu.

Vous n'avez pas besoin de fournir toutes vos conversations, mais les prompts importants doivent être assez précis pour comprendre la contribution du LLM.
```

```
## Utilisation d’un LLM

Outil : ChatGPT
Modèle : GPT-5.6

Utilisation :
- aide pour comprendre print() et return ;
- proposition d'une première version de count_vowels() ;
- aide au débogage du menu principal.

Prompts principaux :
1. "Comment écrire une fonction Python qui compte les voyelles sans utiliser de
liste ?"
2. "Pourquoi mon menu while ne s'arrête pas quand je tape 0 ?"

Modifications personnelles :
- j'ai changé les noms des variables ;
- j'ai réécrit la boucle de count_vowels() ;
- j'ai ajouté la gestion d'une option incorrecte.

Vérification :
- j'ai testé chaque fonction séparément ;
- j'ai comparé les résultats aux résultats attendus.
```

Si aucun LLM n’a été utilisé :

```
## Utilisation d’un LLM

Aucun LLM n’a été utilisé pour ce projet.
```

L’utilisation d’un LLM n’est pas pénalisée en elle-même. En revanche, la déclaration est obligatoire et vous devez être capable d’expliquer le code que vous rendez.